Le script audit.sh

Rapport du code, expliqué bloc par bloc

Linux Server Security Audit Tool

Projet d'apprentissage en cybersécurité

Auteur : Khalil Azzouz • Langage : Bash

Khalil Azzouz

Le script audit.sh expliqué

Le script est écrit en **Bash**, car ses contrôles sont déjà des commandes Linux (`stat`, `awk`, `grep`, `ss`, `find`) : Bash ne fait que les enchaîner et décider « sain » ou « faille ». Aucune dépendance, il tourne sur tout serveur Linux, et reste en **lecture seule** — il regarde l'état du système mais ne le modifie jamais.



Le script audit.sh dans l'éditeur : une fonction par contrôle.

1. En-tête et mode strict

```
#!/usr/bin/env bash
set -uo pipefail
```

- `set -u` : erreur si une variable n'est pas définie (évite les fautes de frappe).
- `set -o pipefail` : une erreur dans un pipe n'est pas masquée.
- On évite `set -e` : dans un audit, une commande qui « échoue » (ex. `grep` sans résultat) est souvent un résultat normal.

2. Les couleurs du terminal

```
if [[ -t 1 ]]; then
    GREEN=$'\e[32m'; RED=$'\e[31m'; YELLOW=$'\e[33m'; RESET=$'\e[0m'
else
    GREEN=""; RED=""; YELLOW=""; RESET=""
fi
```

`[[ -t 1 ]]` teste si la sortie est un vrai terminal. Si oui, les couleurs ANSI sont activées ; si la sortie est redirigée vers un fichier, elles sont désactivées pour ne pas polluer le texte.

3. Le cœur : record() et le score

```
record() {
    local statut="$1" gravite="$2" titre="$3" detail="$4"
    if [[ "$statut" == "FAIL" ]]; then
        case "$gravite" in
            critique) SCORE=$((SCORE - 20)) ;;
            moyen)    SCORE=$((SCORE - 10)) ;;
            faible)   SCORE=$((SCORE - 5))  ;;
        esac
    fi
    printf '  %-6s %-9s %s\n' "$statut" "$gravite" "$titre"
    ROWS+=("| $statut | $gravite | $titre | $detail |")
}
```

Fonction réutilisée par tous les contrôles. Elle (1) ajuste le score selon la gravité si le statut est FAIL (score de départ 100 : critique −20, moyen −10, faible −5), (2) affiche une ligne colorée alignée — `%-6s` et `%-9s` fixent la largeur des colonnes — et (3) stocke une ligne de tableau Markdown pour le rapport.

4. Les six contrôles

#	Fonction	Commande clé	Faille détectée
1	`check_file_permissions`	`stat -c '%a'`	/etc/shadow trop permissif
2	`checkUID0`	`awk -F: '$3==0'`	compte UID 0 caché (admin2)
3	`checkSSH`	`grep PermitRootLogin`	SSH root autorisé
4	`checkUFW`	`ufw status`	pare-feu inactif / absent
5	`check_open_ports`	`ss -tuln`	ports exposés (informatif)
6	`check_world_writable`	`find / -perm -0002`	fichier modifiable par tous

Chaque fonction gère le cas où un service est absent (via `command -v` ou un test de fichier) et marque alors le contrôle `N/A`, sans fausser le score — c'est ce qui rend le script robuste, y compris dans un conteneur minimal.

Exemple : contrôle 2 (comptes UID 0)

```
comptes=$(awk -F: '$3==0 {print $1}' /etc/passwd | paste -sd ',' -)
if [[ "$comptes" == "root" ]]; then
    record "PASS" "-" "Un seul compte UID 0" "root uniquement"
else
    record "FAIL" "critique" "Comptes UID 0 multiples" "Trouves : $comptes"
fi
```

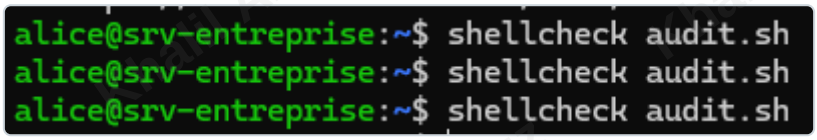
`awk -F:` découpe aux « : », `$3==0` garde les lignes où l'UID vaut 0, `$1` donne le nom. Tout autre résultat que `root` seul est signalé comme une faille critique.

5. Affichage et rapport

`print_header` écrit le titre et les en-têtes de colonnes ; `print_summary` affiche le score final avec une barre de progression (20 blocs, 1 bloc = 5 points), verte / jaune / rouge selon le niveau. `generate_report` écrit toutes les lignes stockées dans `audit_report.md` (tableau Markdown + machine, date, score).

6. Qualité du code

Le script passe **ShellCheck**, le linter de référence pour Bash, **sans aucun avertissement** — preuve d'un code propre (guillemets corrects, variables bien utilisées).



ShellCheck ne renvoie aucun avertissement (sortie vide = code propre).

7. Résultats



État vulnerable : 50/100, les 4 failles détectées.



État propre : 100/100, tout en PASS.

> **Prochaine étape.** Tester le script dans un conteneur Docker minimal : sans SSH ni pare-feu, les contrôles concernés doivent passer en N/A sans fausser le score.